RETAIL-DATA-AGENT · STEPS 1–2 AND 5 DONE, STEP 3 ALMOST · AS OF 2026-11-01 (REPORTS SAVED, LISTED AND DELETED)

Retail Data Agent Map

How one question travels from your terminal to BigQuery and back, and why each file in the repo exists. See also why run_sql has a factory, how each user sees only their brands, how the parser rules stop a bad query, and how analysts' past work would guide the agent (design only).

🟩 What you see 🟧 Agent and LLM 🟦 Data path 🟪 Config and data files

One question, end to end

Each box is a file. The agent and the LLM go back and forth until the model has enough data to answer. A normal question takes 2 LLM calls.

`cli.py`
Started as `retail-agent --user levis_manager`; the user and their brands come from `users.yaml` (`users.py`). You type *"Top 5 brands by revenue last month?"*. Prints only the ANSWER block (`make run`), or a clear message if the model is busy. With `--verbose` (`make run_logs`) it first prints every SQL the model tried, each with ✅ ran or ❌ and the reason, and for a query that ran, the SQL sent to BigQuery after the gateway's rewrites (pseudonyms, brand scope).

↓ `app.ask(question)`

`app.py → App.ask, App.confirm`
Sends the question to the agent and collects the answer plus every SQL statement run during the turn. Keeps the conversation under one thread id, so follow-up questions see the earlier turns. When a tool pauses the turn (a delete waiting for a yes), it returns the confirmation text instead, and `App.confirm` resumes the turn with what the user typed.

↓ `invoke`

`agent.py`
LangChain `create_agent`: the loop between the model and the tool, with `CallBudget` middleware for the 5-call limit and a checkpointer that saves the conversation.

↓ `model ⇄ tool`

`llm.py`
Builds the chat model from `LLM_PROVIDER:LLM_MODEL`, with a 30 s timeout and 1 retry. Gemini by default; Claude when you switch it on.

↓ `tool call: run_sql(sql)`

`tools.py → run_sql, save_report, list_reports, delete_reports`
`run_sql` hands the SQL to the gateway and turns the result into a compact text table for the LLM (`result_format.py`), with a note when rows were cut off or none matched. A rejected query comes back as `Error: …`. `save_report` (step 5) saves a finished report to `reports.py`'s SQLite file, with the owner, conversation, question and SQL filled in by the code, and its result is the final answer. `list_reports` lists the asking user's own reports, optionally those mentioning some text or made in this conversation.

↓ `gateway.run(sql)`

`query_safety/gateway.py → QueryGateway`
The only path to the data. Logs the SQL, runs the guard (`query_safety/step1_guard.py`: one SELECT, only the 4 tables), lets the query read only the columns in `schema.yaml` so no personal data gets out (`query_safety/step2_personal_data/columns.py`), lets age, city and other demographic columns appear only in totals of 5+ customers (`query_safety/step2_personal_data/small_groups.py`), shows customer ids only as pseudonyms such as `cust_72ac84091163` (`query_safety/step2_personal_data/customer_pseudonyms.py`), limits it to the user's brands (`query_safety/step3_brand_scope/brand_scope.py`), rejects a query whose dry-run estimate is over the byte limit, caps rows. See one query's full journey, the parser rules and brand scoping.

↓ `database.execute(sql, max_rows)`

`db/bigquery.py`
Runs the query in BigQuery, billed to `GCP_PROJECT`, never more than `MAX_BYTES_BILLED`. Follows the `Database` protocol in `db/base.py`.

↓ `job`

`bigquery-public-data.thelook_ecommerce`
orders · order_items · products · users. A typical question scans 3–7 MB.

The agent loop

1. LLM reads the question, the earlier turns, the schema and the rules, then writes SQL.
2. `run_sql` runs it and returns rows as a table. Independent queries go out together in one reply and run at the same time.
3. LLM reads the rows and writes the answer. For a report, it writes the report into `save_report` instead, which saves it and shows it as the answer. Asked about saved reports, it calls `list_reports`, whose list is the answer.

If the SQL fails or the guard rejects it, the error goes back to the LLM so it can fix it. A question gets at most 5 LLM calls; the 5th has no tools, so it answers from the results it already has.

What you see in the terminal for that question:

```
> What were the top 5 brands by revenue last month?
Working...

── SQL ──────────────────────────────────────
SELECT p.brand, ROUND(SUM(oi.sale_price), 2) AS revenue
FROM `bigquery-public-data.thelook_ecommerce.order_items` AS oi
JOIN `bigquery-public-data.thelook_ecommerce.products` AS p ON oi.product_id = p.id
WHERE oi.status NOT IN ('Cancelled', 'Returned')
  AND oi.created_at >= TIMESTAMP(DATE_SUB(DATE_TRUNC(CURRENT_DATE(), MONTH), INTERVAL 1 MONTH))
  AND oi.created_at < TIMESTAMP(DATE_TRUNC(CURRENT_DATE(), MONTH))
GROUP BY p.brand ORDER BY revenue DESC LIMIT 5

── ANSWER ───────────────────────────────────
1. Diesel: $8,024.59      2. True Religion: $7,671.26
3. Calvin Klein: $7,036.47   4. Carhartt: $6,073.56   5. 7 For All Mankind: $6,024.97
Revenue = sum of sale_price, excluding Cancelled and Returned items.
```

Meanwhile `observability.py` and `query_safety/gateway.py` write each step to `logs/retail-agent.log`: each LLM call with its duration, each SQL with rows and bytes.

At startup, once

Before the first question, `app.py` puts the pieces together. `build_app` creates the real model and database; `assemble` wires them. Tests call `assemble` directly with fakes.

`.env → config.py`
Model, keys, GCP project, row and byte limits. `.env` is never committed.

`schema.yaml → schema.py`
The 4 tables, cleaned up, with short notes. Names, emails and addresses are left out on purpose.

`prompts/system.md`
The rules: answer only from query results, compute dates in SQL, state the revenue definition.

↓ filled in together ↓

system prompt
Rules + schema + SQL dialect, sent with every LLM call. The agent never has to look up the schema itself.

Every file and why it exists

FILE	WHY IT EXISTS	GROWS IN
Application code · src/retail_agent/		
🟩 `cli.py`	The terminal chat. Loads `.env`, sets up logging, prints CONFIRM and ANSWER blocks, turns failures into plain messages. `--user` picks who is asking. A plain run shows only the answer; `--verbose` adds SQL blocks for every attempt, with ✅ or ❌ and the reason, plus the SQL sent to BigQuery, from `TurnResult.sql` (`SqlRun(sql, error, sent_sql)`). `sent_sql` rides on the `run_sql` tool's artifact, so the model never sees it. A CONFIRM block waits for a typed answer; Ctrl-D counts as no.	–
🟧 `app.py`	Wires everything in one place. `build_app` picks the real parts, `assemble` connects them, `App.ask` returns the answer and SQL, or a confirmation to show; `App.confirm` resumes a paused turn. One `App` is one conversation.	each step
🟧 `agent.py`	Creates the LangChain agent. Loop rules attach here as middleware: `CallBudget` limits a question to 5 LLM calls and makes the last one answer.	step 5
🟧 `llm.py`	One function that builds the chat model from config, so switching providers never touches other code.	–
🟦 `tools.py`	The tools the LLM calls: `run_sql` (rejected queries come back as `Error: …`) `save_report`, which saves a report and ends the turn with it, so saving costs no extra LLM call; `list_reports`, whose list is the answer too; and `delete_reports`, which shows the exact list, pauses the turn, and deletes those reports only after a typed `yes`.	–
🟦 `reports.py`	The report store: a SQLite file at `REPORTS_PATH` (default `data/reports.sqlite3`), created on the first save. Each row has the owner, conversation id, time, question, the SQL behind the numbers and the report text. `find` reads them back, always filtered on the owner in the SQL, by text (case-insensitive contains) or conversation. `delete` removes the ids a `find` returned, also filtered on the owner.	–
🟦 `result_format.py`	How results are shown to the LLM: a compact table, plain dates and numbers, a note when rows were cut off, and "no rows" said in words.	–
🟦 `query_safety/gateway.py`	The single door to the data. Every safety rule lives here, so nothing can go around it: the guard, the personal-data check, the small-groups check, the customer pseudonyms, the brand scope, the byte limit, the row cap.	step 3
🟦 `query_safety/step1_guard.py`	Parses each query with sqlglot before it runs: exactly one SELECT, no query parameters or system variables, reading only the allowed tables, no table functions. Fails closed if it can't parse.	step 3
🟦 `query_safety/step2_personal_data/columns.py`	Lets a query read only the columns listed in `schema.yaml`, traced through aliases, joins and subqueries with sqlglot. Also rejects `*` (except `COUNT(*)`) and whole rows used as values. Names, emails and addresses aren't listed, so they can't be read.	–
🟦 `query_safety/step2_personal_data/small_groups.py`	Lets the columns tagged `aggregate_only` in `schema.yaml` (age, gender, city, state, country, traffic source) appear only inside totals, and adds `HAVING COUNT(DISTINCT customer id) >= 5` to those totals so BigQuery drops groups under 5 customers. A listing, or ARRAY_AGG / STRING_AGG, is rejected. See small groups.	–
🟦 `query_safety/step2_personal_data/customer_pseudonyms.py`	Rewrites each query that reads a customer table so the id is a salted hash (`cust_72ac84091163`), for every user. The same customer always gets the same pseudonym, so joins and follow-ups still work. The salt reaches BigQuery only as a query parameter.	–
🟦 `query_safety/step3_brand_scope/brand_scope.py`	Rewrites each query so every table it reads holds only the user's brands. A user with every brand (the CEO) skips this step.	–
🟦 `query_safety/step3_brand_scope/order_item_count.py`	For a brand manager, recounts `orders.num_of_item` from their own items, so it no longer counts other brands' items.	–
🟦 `db/base.py`	The `Database` protocol (`execute`, `estimate_bytes`), `QueryResult` and `QueryError`. What any database must offer to plug in.	ticket 05
🟦 `db/bigquery.py`	The BigQuery implementation: billing project, byte cap, dry-run byte estimate, rows and bytes processed. Sorts errors into query problems (back to the model) and infrastructure ones.	ticket 05
🟪 `config.py`	Typed settings read from the environment, with defaults.	as settings are added
🟪 `users.py`	Reads `users.yaml` with pydantic and returns the `--user` with their brands. A bad file or unknown user stops the CLI at startup.	–
🟪 `schema.py`	Reads `schema.yaml` and renders it for the prompt.	ticket 05; step 3
🟧 `observability.py`	Logs every LLM call with its duration or error, so a slow or busy model is visible.	step 6
🟪 `__init__.py, db/__init__.py`	Mark the folders as Python packages.	–
Data and prompt · edited without touching code		
🟪 `schema.yaml`	What the agent knows about the data: tables, columns, types, notes such as the revenue definition. Also the column allow-list: a column not listed here can't be queried.	–
🟪 `users.yaml`	Who can use the agent and which brands each one sees; `all` for the CEO.	as users are added
🟪 `prompts/system.md`	The agent's instructions, as a template. Persona and tone change here.	as needed
Setup		
🟪 `.env.example`	Every setting documented. Copy it to `.env` and fill in your keys.	as settings are added
🟪 `pyproject.toml, uv.lock`	Dependencies and the `retail-agent` command. Anthropic is an optional extra.	–
🟪 `.python-version, .gitignore`	Pins Python 3.12. Keeps `.env`, `logs/`, `data/` (saved reports) and the virtualenv out of git.	–
Tests · run offline, no keys needed		
🟩 `tests/fakes.py`	A scripted fake LLM that records what it was sent, and a fake database with canned rows and byte estimates.	each step
🟩 `tests/test_reports.py`	A report turn (two parallel queries, then `save_report`) saves one row with the right owner, conversation, question and SQL in 2 LLM calls; a rejected query is left out; a normal question saves nothing. Listing never shows another user's reports, searches title, question and text ignoring case, and answers in 1 LLM call. Deleting shows the exact list, deletes only after `yes` (anything else cancels), never touches another user's reports, deletes only the ids that were shown, and confirms two deletes one at a time.	step 5
🟩 `tests/test_turn.py`	Drives full questions through `assemble` with the fakes: answer, SQL, prompt, logs, failures, call budget, follow-ups.	each step
🟩 `tests/harness.py`	Shared helpers: builds an app from the fakes and reads what the model saw from a tool.	each step
🟩 `tests/test_step1_guard.py`	Each guard rule and the byte limit, through the whole app: rejected queries never reach the database.	step 3
🟩 `tests/test_step2_personal_data.py`	Personal columns, `*` and whole rows are rejected before the database; schema columns, joins and `COUNT(*)` still run.	step 3
🟩 `tests/test_step2_small_groups.py`	Listings of tagged columns are rejected (also through subqueries, WITH steps, windows and ARRAY_AGG); totals get the HAVING on the right SELECT; queries without tagged columns are unchanged; the minimum comes from `MIN_GROUP_SIZE`.	–
🟩 `tests/test_step2_customer_pseudonyms.py`	No query reads a real customer id, for the CEO or a brand manager; only the copies a query reads are added; the salt goes to BigQuery as a parameter, never in the SQL.	step 3
🟩 `tests/test_step3_brand_scope.py`	Tricky queries can't read a table outside the brand filter; a user with every brand isn't filtered; the gateway runs the scoped SQL.	step 3
🟩 `tests/test_step3_order_item_count.py`	A brand manager's orders recount `num_of_item` from their own items.	–
🟩 `tests/test_users.py`	Loading `users.yaml`: brand lists, `all`, unknown users and broken entries.	–
🟩 `tests/test_bigquery_errors.py`	Which BigQuery errors go back to the model and which stop the question.	–
Docs and planning		
🟪 `docs/plan.md`	The brief, client answers, the 7-step plan and every design decision with its rejected alternatives.	each step
🟪 `docs/hld.md`	The high-level design: architecture and request-flow diagrams.	each step
🟪 `docs/specs/step-1-core-loop.md`	Step 1 in detail: user stories, decisions, test cases.	–
🟪 `tasks/01–07`	Step 1 split into tickets. 01–04 and 07 (sign-off) are done. 05 moved to step 4 (optional) and 06 to step 7.	–
🟪 `README.md`	Stub for now. Ticket 06 turns it into the setup guide.	ticket 06
🟪 `logs/retail-agent.log`	Created when you run the agent. Not committed.	step 6

Where later steps plug in

The structure exists so each step adds to one place instead of rewriting the rest.

Query gateway

- Done in step 2: only single SELECTs on the 4 tables; dry-run byte check
- Done in step 3: every query limited to the user's brands
- Done in step 3: only `schema.yaml` columns, no `*`, so no personal data
- Done in step 3: customer ids only as salted-hash pseudonyms, for every user
- Done in step 3: demographics only in totals, groups under 5 customers hidden
- Step 3 next: a safety-net check on returned rows

Agent middleware

- Done: at most 5 LLM calls per question, the last one answers (and says a report can't be saved then)
- Done in step 5: a delete pauses the turn (LangGraph `interrupt`) until the user types yes

Database protocol

- Ticket 05: check `schema.yaml` against live tables at startup
- Another engine = another adapter, nothing else changes

Tests via assemble

- Fake LLM records exactly what it was sent
- Step 3 uses this to prove personal columns never reach the database or the LLM